

車牌辨識系統之實作

學生:林郁佐

指導老師:林峻鋒

南華大學資訊工程學系

摘要

汽車已經成為現今的主要交通工具，而車牌就猶如汽車的身份證，想要對汽車進行有效的管理從車牌著手最為實際，本篇專題主要是針對汽車的車牌，做相關的影像處理及辨識的實作，進而可以應用到大樓的門禁系統、停車場的管理系統，甚至是贓車的查詢系統。本篇專題實作的辨識流程分為兩大部份，第一部份是將影像中的車牌區域擷取出來，以供後面辨識使用；第二個部份是將前面擷取出的車牌影像中的號碼依序切割出來並加以辨識，最後將辨識結果顯現出來。

關鍵字：車牌、辨識

1. 研究動機

由於汽車已經變成人們主要的交通工具，車輛的管理顯得更為重要，傳統用人工進行管理的方法不符合經濟成本，於是想藉由對車牌的辨識，應用於對車輛的管理。並藉由實作空間域的影像處理，瞭解影像處理的原理。

2. 研究目標

藉由車牌辨識的應用，可以大大地減輕管理人員的負荷，在系統技術成熟的情況下，也可以提高系統的自動化程度。

3. 系統限制

專題的實驗對象為數位相機拍攝的車牌照照片，拍攝距離約為一到二公尺，照片的大小為640*480的JPEG影像。

4. 系統流程

在車牌擷取部份，先將車牌轉成灰階影像以便後續處理，之後利用Prewitt遮罩留下明顯的垂直邊緣，接著進行二值化，由於影像目前仍然留有許多小雜點，於是進行兩次的中值濾波，之後將影像膨脹處理讓車牌區域連結成一塊以便後續的篩選，再對影像進行侵蝕來清除細小的非車牌區塊，最後經過標籤化及條件篩選將車牌區塊擷取出來。

在車牌辨識部份，先將擷取出來的車牌影像轉換成灰階，再將它進行一次中值濾波，之後把它二值化，這裡的二值化跟車牌擷取時進行的二值化不同，後面會再詳細說明，之後將車牌字元以外的部份切除，再將字元各自切割出來，再將字元進行正規化，最後將正規化以後的字元進行辨識。

系統流程圖如下：

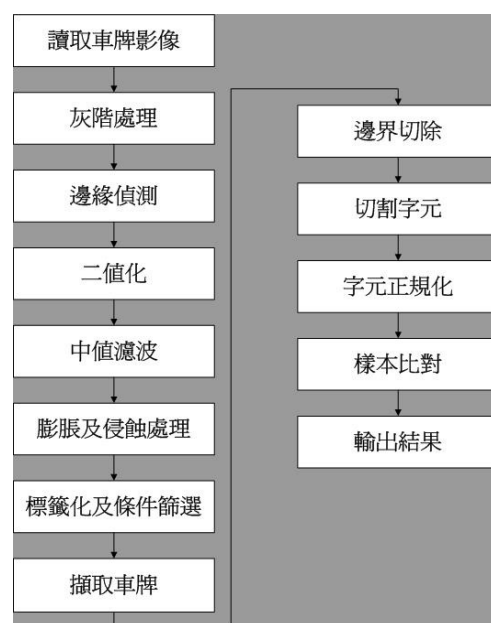


圖 1. 系統流程圖

5. 研究工具

硬體環境：

CPU: 1GHz

RAM: 256MB

軟體環境：

程式開發軟體：Microsoft Visual Studio .NET 2005

作業系統：Windows XP SP2

6. 實作步驟

6.1 車牌擷取

第一部份利用邊緣偵測找出車牌區域，擷取出車牌區域，再由第二部份進行辨識。

6.1.1 灰階處理

車牌的影像格式為 RGB，為了方便後續的處理，於是轉成只剩下亮度的灰階影像，灰階轉換的公式如下：

$$\text{Gray}=0.3R+0.59G+0.11B$$



圖 2. 原始影像



圖 3. 灰階影像

6.1.2 Prewitt 遮罩

進行 Prewitt 運算的目的是找出影像內明顯的垂直邊緣點，所謂的邊緣指的是灰階值落差極大的兩點，而車牌區塊內的字元部份，會形成許多密集的邊緣，這個特性讓我們能夠對車牌區域進行尋找。我們利用 Prewitt 來當作偵測垂直邊緣的方法，Prewitt 遮罩的係數如下：

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

表 1. Prewitt 遮罩係數

將經過遮罩的運算值乘上一個增益值，本專題的增益值為 0.65，讓它成為判斷的門檻值，將超過門檻值的點設成黑點，這樣影像裡的垂直邊緣就會被區分出來。



圖 4. 經過 Prewitt 運算後

6.1.3 二值化

不同的門檻值對二值影像的影響極大，若採用固定門檻值的二值化方法，則不見得每張車牌影像都能得到最適合後續處理的影像。為了這個問題，本專題使用參考資料[1]裡面所提到的二值化方法，先得到影像中最大跟最小的像素值，其總合的一半即為初始的門檻值 T ，接著將影像裡大於此門檻值的像素值加總起來取其平均值 O_{sum} ，小於此門檻值的像素也同樣加總起來取得其平均值 B_{sum} ，再把 O_{sum} 跟 B_{sum} 加起來除以二得到 T_n ，若 T_n 不等於 T ，則將門檻值 T_n 設為新的門檻值 T ，直到 T_n 等於 T 為止，這樣取得的 T 即是當作二值化依據的門檻值。

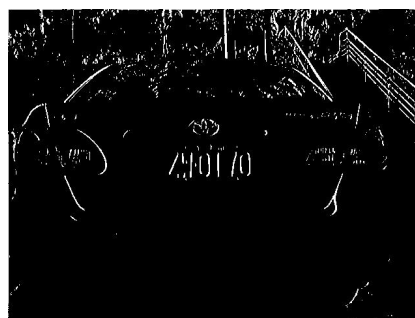


圖 5. 經過二值化以後的影像

6.1.4 中值濾波

影像處理到現在，畫面裡還有許多小的雜點，可能會影響到後面的辨識，於是需要使用消除雜訊的方法來去除畫面中的小雜點，常見的消除雜訊方法如移動平均法，由於使用移動平均法易使影像變得模糊，於是本專題使用較不會使影像變得模糊的中值濾波，使用兩次中值濾波來去除畫面中的雜點。

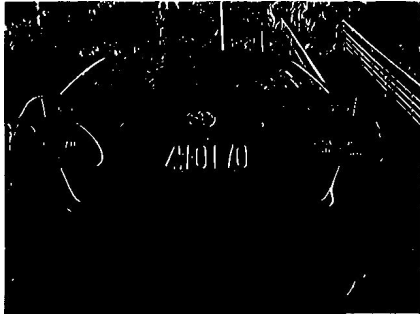


圖 6. 經過一次中值濾波



圖 7. 經過二次中值濾波

6.1.5 膨脹及侵蝕

膨脹的目的是將車牌區域的垂直邊緣點，連結成一個區塊，以便後續車牌區塊篩選；侵蝕的目的是將影像內過小的雜點去除，讓它們不至於干擾到後續的處理。

本專題進行膨脹及侵蝕處理的結構元素為寬 20 高 2。



圖 8. 經過膨脹處理

經過膨脹處理以後可以發現，車牌區域的垂直邊緣點已經連結成一塊，接著要進行侵蝕處理將影像裡的雜點去除。

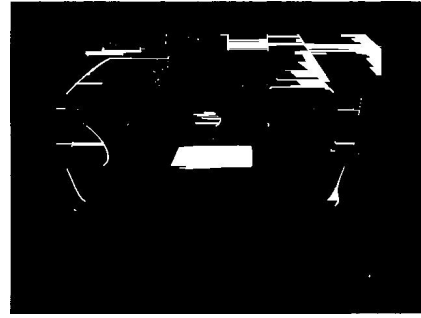


圖 9. 經過侵蝕處理

為了更仔細地清除雜點以及讓車牌區塊更為完整，繼續再使用各一次的侵蝕及膨脹，此時所用的結構元素與之前相同。

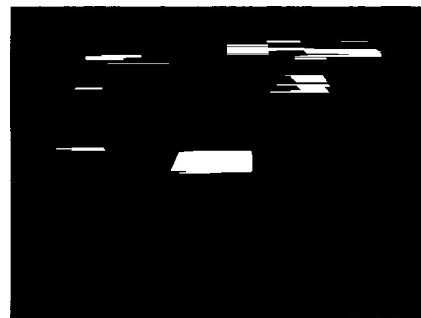


圖 10. 再經過各一次侵蝕及膨脹處理

6.1.6 標籤化

將畫面裡的白色區塊標籤化，以便後續用來篩選出可能的車牌區塊，標籤化的演算法如下，主要是根據參考資料[4]裡面提到的演算法來實作，由於這個演算法經過實作以後發現，若白色區塊的周圍有突出的地方，將會使得原本應該是同一區塊的區域被標記成兩個不同的標籤，於是本專題再加上幾個判斷條件，來避免這樣的情況發生。

標籤化的演算法如下：

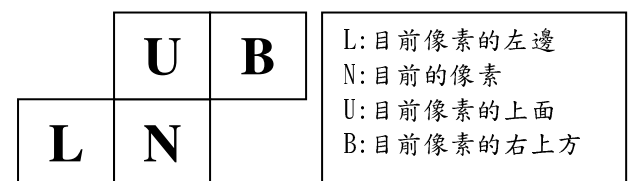


圖 11. 演算法示意圖

```

for(影像裡的每一列){
  for(影像裡的每一行){
    N=目前的像素且為白點
    if(如果 L、U、B 皆未有標籤)
      N=新的標籤
    else if(B 已標籤且 L、U 皆未標籤)
      N=B 的標籤;
  }
}
  
```

```

else if(U 已標籤且 L 未標籤)
    N=U 的標籤;
else if(L 已標籤且 U 未標籤)
    N=L 的標籤;
else if(L、U 皆有標籤且標籤相同)
    N=U 的標籤;
else if(L、U 皆有標籤且標籤不同)
    N=U 的標籤;
    將此條水平線目前像素之前
    跟 L 一樣的標籤設成 U 的標籤
}
}

```

經過標籤化的處理，可將影像裡相連結的白點連成一個區塊，而可能包含車牌的區塊將在下一步驟經過條件判斷篩選出來。

6.1.7 篩選車牌區塊

將已經標籤化的區塊用一些條件來篩選出含有車牌的區塊，篩選條件如下：

條件一：

由於含有車牌的區塊大部份都有一定的面積，本專題實驗的結果為至少在 1600 pixels 以上。

條件二：

由於車牌的長寬比例有一定的範圍，比例不在此範圍內的區塊即可排除，本專題篩選用的比例為寬除以長在 1.5~5.0 之間。

條件三：

經過上面兩個條件的篩選以後，畫面裡所剩的區塊已經不多，而車牌區塊下方的干擾物通常不多，所以車牌區塊大多為最下方的區塊。

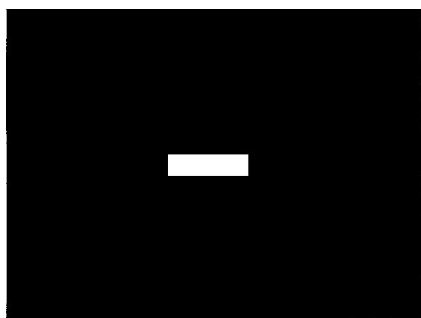


圖 12. 經過篩選後的區塊

6.1.8 擷取車牌

將挑選出的車牌區塊擷取出來，就可以得到車牌部份的影像。



圖 13. 擷取出的車牌影像

6.2 車牌辨識

將擷取出的車牌區塊切除非字元的邊界，並逐序切割出字元，再經過正規化以後進行樣本比對，顯示出辨識結果。

6.2.1 前處理

擷取出來的車牌照照片一樣經過灰階處理以便後續處理。



圖 14. 灰階處理

接著再進行一次的中值濾波去除雜訊。



圖 15. 經過一次中值濾波

最後將影像二值化以便後續處理，這裡的二值化跟之前擷取車牌部份所用的二值化不同，由於現在的影像已沒有車體顏色跟背景的影響，於是本專題使用均值二值化，如此可以減少字體部份的破損。



圖 16. 經過均值二值化以後的車牌影像

6.2.2 邊界切除

辨識的目標為車牌的字元部份，所以要將字體以外的部下邊界跟左右邊界來進行，採份去除，本專題邊界切除用的演算法為參考資料[1]裡面提到的方法，由於在實作上遇到有些問題，於是再加上一些判斷條件來解決實作上遇到的問題。

一. 切除上下邊界：

將影像從中心點往左右各延伸 30 個 pixels，接著分別往上下水平搜尋，一旦發現整條水平線全為

白點或者是白點個素大於 90%，則將此條水平線標記為切除線，並停止搜尋，如此便能成功將上下邊界切除。另外，有些字體由於破損嚴重，可能在距離中心點過近的地方就判斷為切除線，於是加上一個判斷條件，切除線的距離需大於高度除以六，否則繼續搜尋，如此便能將車牌的上下邊界切除。



圖 17. 切除上下邊界

二. 切除左右邊界:

從左邊往右邊搜尋每條垂直線，一旦發現整條垂直線白點個素少於 95%，則停止搜尋並將此條垂直線標記為切除線，右邊界的切除線搜尋方法亦是如此，只是將方向改成由右至左搜尋，如此便能將左右邊界切除。另外，由於有些字體會跟背景相連而導致背景未被切除或是字元被排除的情況，於是在這裡加上一判斷條件，判斷由那邊可以正確地切割出六個車牌字元。若是左右切除的距離小於字元區域最小的長寬比例，則有可能有字元被排除在外，於是將切除距離較短的那邊標記成可以正確切割字元的開始方向；若是左右切除的距離大於最大的長寬比例，則有可能有背景未被切除，於是將切除距離較長的那邊標記成可以正常切割字元的方向，若切除距離在合理的長寬比例裡，則預設正確的切割字元方向為左邊。



圖 18. 切除左右邊界

6.2.3 字元切割

切割字元是依照前一步驟標記的方向往另一邊逐次切割出字元，每次找出字元切割的起點跟終點再進行切割，每次切割完的終點即為下一個字元切割的起點，而切割的終點為起點加上一個字元的寬度，在起點跟終點附近找到黑白像素差距最大的垂直線，即為字元的切割線，重複此步驟直到切出六個字元。由於車牌中間的「-」符號會影響到切割字元的位置，於是必須先判斷符號的位置是在左邊或右邊，判斷是根據頭兩個切割出的字元為英文或是數字來判斷，若至少有一個字元辨識出來為英文字，則符號將緊接在後面，反之則在另一邊，如此可在切割字元時跳過符號造成的影響。另外數字「1」由於字體本身寬度較短所以在碰到「1」時也要做另外的處理，一旦辨識出目前的字元為「1」則下一個字元切割的起點為「1」的切割終點往後找到的第一條不是白色的線，如此便能不被字元「1」影響成功切割出字元。



圖 19. 字元切割線

6.2.4 字元正規化

為了後續與樣本字元比對，必須將切割出來的字元正規化，本專題採用線性正規化，將字元正規化成 15*45 的大小，正規化的方法是以 15*45 的座標推斷欲正規化字元在此大小裡的座標是黑點或白點。



圖 20. 正規化以後的字元

6.2.5 字元辨識

本專題採用的辨識法為樣本比對法，使用的樣本字元如下圖表所示：

0	1	2	3	4	5	6	7
8	9	A	B	C	D	E	F
G	H	I	J	K	L	M	N
O	P	Q	R	S	T	U	V
W	X	Y	Z				

表 2. 字元樣本

經過正規化以後的字元跟樣本字元逐一比對，選出最相似的字元，即是字元的比對結果。

另外，針對幾組較容易辨識錯誤的字元如『0』跟『D』、『B』跟『8』以及『S』跟『9』，判斷比對的相似個數，如果最相似以及第二相似的字元，兩者的差距小於一定範圍，則再進入二次比對。

二次比對只比較重點區域，如『0』跟『D』的左上角及左下角，累計這些區域跟樣本字元裡面的白點，相似的數量多寡判斷這個字元是『0』或者是『D』，經過此項處理更能增加字元辨識的正確率。

7. 實驗結果



圖 21. 系統介面圖

本專題實驗的結果：

實驗車牌影像:131 張。
辨識正確張數:81 張。
辨識率:61.83%。
總共辨識字元:672 個。
辨識正確字元:604 個。
字元辨識率:89.88%。

8. 未來研究方向

本專題的辨識率不盡理想，其原因可能是因為影像的品質包括陰影、角度、亮度的影響，或是影像處理的演算法，專題所使用的一些演算法未必能在每張影像裡得到預期的結果，未來將克服影像品質所帶來的影響，如：傾斜角度的矯正、影像對比度的增強。另外，將會尋找或者自行設計更能達到預期結果的影像處理演算法，待車牌的辨識率達到一定目標以後，將會再搭配相關硬體，如攝影機，以能夠構成一套完整的車牌辨識系統。

9. 參考資料

- [1] 童子倫，“車牌辨識系統”，逢甲大學資訊工程學系，2004
- [2] 鄭惟仁、朱耿佑，“車牌辨識”，大同大學資訊工程學系，2007
- [3] 林仲芬(1995)，「影像辨識技術『第二版』」，全華出版。
- [4] 井上誠喜、八木伸行、林正樹、中須英輔、三谷公二、奧井誠人(2006)
「C 語言數位影像處理」，全華出版。
- [5] 楊武智(1994)，「影像處理與辨識」，全華出版。
- [6] 鍾國亮(2004)，「影像處理與電腦視覺」，東華出版。
- [7] 程式設計俱樂部，
<http://www.programmer-club.com/index.asp>

[8] MSDN，

<http://www.microsoft.com/taiwan/msdn/default.t.aspx>

[9] 井民全，「簡單的數位影像處理『C#篇』」